

Does Agentic Architecture Discovery Reproduce?

An Audit Gate That Blocked Its Own Headline Result

Anonymous Submission

Abstract

Agents already *discover and exploit loopholes in microarchitectural simulators* [1], so we measure evaluation validity rather than design discovery. We release a CHIA audit loop that separates four variance sources an aggregate score hides—repeated-run, generation-seed, prompt and trace—gating publication on both a pre-registered threshold and a double-blind annotation layer whose annotators are independent models. Run in the official `chia-champsim` image, the loop *failed* its own gate: cross-seed CV 78.16% against a 5% threshold while repeated-run CV is 0.0000% over 18 runs, so the publish gate reads `BLOCKED`. Two candidates differing only in a struct name and indentation carry distinct source, design and binary digests yet bit-identical cycles on all three traces: the seed moved the artifact, not the design. Ranking is unstable (top-1 0.667, Kendall $\tau = 0.556$). Annotation gives $\kappa = 0.859$ on specification diffs but 0.667 on real generated C++, where both auditors agree and are both wrong about a measured simulator escape. We report the negatives instead of moving the thresholds.

1 Introduction

Architecture 2.0 pipelines (ArchAgent [1], CHIA [2], QuArch) let agents propose and evaluate designs with no human in the loop. The risk is not unclaimed: ArchAgent reports its pipeline discovered and exploited a loophole in a popular microarchitectural simulator, inflating reported performance without changing hardware logic [1]. Software-side evaluation shows the same fragility—all top-three Terminal-Bench 2 submissions cheated via harness vulnerabilities [3], OpenAI deprecated SWE-bench Verified and retracted SWE-bench Pro [4], and > 25.7% of 168 reviewed benchmarks carry critical defects [5].

What is missing is an instrument, not another warning: a loop that runs against a real simulator, separates the variance sources an aggregate score hides, and refuses to publish when the separation fails. We contribute that loop plus what it said about us—a `BLOCKED` verdict on our own headline claim (§3.1), evidence that generation seeds move artifacts without moving designs, trace-dependent reordering (§3.2), and a failure mode that text-reading auditors cannot catch (§3.4).

2 The Audit Loop

Prepare. Pin git SHA, interpreter and image digest; load the pre-registered configuration. Thresholds are frozen before any measured cell exists, which is what makes the §3 verdicts binding rather than negotiable [6].

Run. Execute a $|\text{seeds}| \times |\text{prompts}| \times |\text{traces}|$ grid, n repeats per cell, through CHIA’s `ChampSimNode` in the official image, recording the candidate digest, delivered binary digest and raw simulator JSON per cell. **Audit.** Two annotators classify candidate against reference using E0–E4 (Table 1). The payload is built from an allowlist of `id`, `design`, `reference`; the script refuses to run if any expected label survives. We report the deterministic in-loop self-test and the independent-model layer separately and never merge them. **Act.** Emit the scorecard and gate: `publishable` requires reproducibility PASS *and* audit PASS. A nullity check compares each candidate with a no-op reference measured on the same trace and instruction budget.

Class	Description
E0	Equivalent
E1	Parameter mislink
E2	Direction error (LRU↔MRU)
E3	Missing component
E4	Added or fabricated component

Table 1: Taxonomy. §3.4 reports a failure mode it cannot express.

Setup. `ghcr.io/ucb-bar/chia-champsim` at digest `sha256:610951d3...`, `ChampSim 164fdb1e...`, Python 3.10.19, Ray 2.54.0, one `c2d-standard-8 VM (us-east1-b)`; 34 unit tests pass inside the image. Three DPC-4 traces (`fotonik3d`, `ligra BFSCC`, `imagemagick`) with SHA-256 in `results/grid_v6/provenance.json`, 1M warmup + 4M simulated instructions, L2C. Candidates are three real LLM-generated prefetcher modules retained with raw prompts and responses, loaded through directory mode; seeds and prompts select the artifact and the simulator is never handed a seed. Calibration is 5 gold + 5 adversarial specification cases plus 4 semantic cases built from the generated C++ itself. Pre-registered metrics: cross-seed CV < 5%; repeated-run CV, prompt spread, trace CV, top-1 stability and leave-one-trace-out Kendall τ reported; annotation gate $\kappa \geq 0.7$.

3 Results

3.1 The gate blocked our own headline claim

Across 3 designs $\times$ 3 traces $\times$ 2 repeats the loop reports cross-seed CV **78.16%** against the 5% threshold, verdict NON-REPRODUCIBLE, publish gate BLOCKED. Repeated-run CV is **0.0000%** over 18 runs: the simulator is deterministic, and determinism is not reproducibility.

The seed axis explains the gap. **s1** and **s3** differ only in the struct name and two lines of continuation indentation, yet each carries a different source, design and binary digest while all three traces return bit-identical cycles. **s2** keeps a fill history, is 10.6% faster than the no-op reference on BFSCC, and reproduces the no-op cycle count exactly on imagick. The seed moved the artifact and not the design, and nullity is a per-trace property—our cell-level checker conflates the two, which we record as a protocol defect.

3.2 Trace choice reorders the candidates

Trace CV is 163.33%, top-1 stability 0.667, mean Kendall $\tau = 0.556$: the ranking is not stable under the trace subset that produced it. The no-op/next-line control pair separates by 23.15% on BFSCC but only 0.42% on fotonik3d, so a ranking that moves only on fotonik3d is weak evidence and the grid must be read per trace, not averaged.

3.3 Independent annotators, and where they agree and are wrong

On the 10 specification cases the two models give Cohen’s $\kappa = \mathbf{0.859}$ (0.900 agreement, verdict accuracy 10/10 for both, error-class exactness 7/10 and 8/10). Rebuilding the cases from real generated C++ instead of dictionary diffs drops κ to **0.667**—below our own 0.7 gate, so the annotation layer is judged red on the harder surface. The same run showed our calibration set is partly unidentifiable: two cases carry expected labels that cannot be derived from the payload the annotator sees.

3.4 A nullity escape both auditors missed

Compile-and-run is not evidence of a design. Against a cold no-op reference measuring 1,138,748 cycles, two *distinct* binaries—a generated stride prefetcher and a repaired aggressive-offset candidate—return exactly 1,138,748, while every genuine prefetcher differs (1,129,110 / 1,129,706 / 1,132,568) and the next-line control returns 1,129,305. Two of 17 screened designs therefore compile, execute, emit metrics and do nothing; one is not a lazy stub but a plausible stride prefetcher whose

issue path never fires. Any verifier accepting “builds, runs, produces numbers” passes both.

Both independent annotators classified this case **not_equivalent/E4**; measurement says **equivalent**. They agree with each other and disagree with the simulator—a correlated failure, not independent noise. E0–E4 also has no slot for “claims to work, measurably does nothing”: it audits pairwise specification difference, whereas the escape is a claim-to-measurement gap. What catches it is the nullity check, not the taxonomy. Reading a candidate is a poor proxy for running it, and more textual auditors buy agreement rather than independence when they share a failure mode.

Metric	Gate	Measured
Cross-seed CV (max)	< 5%	78.16%
Repeated-run CV (max)	report	0.0000%
Prompt spread (max)	report	not measured
Trace CV (max)	report	163.33%
Trace top-1 stability	report	0.667
Mean Kendall τ	report	0.556
κ , spec cases	≥ 0.7	0.859
κ , semantic cases	≥ 0.7	0.667
Publish gate	both pass	BLOCKED

Table 2: Pre-registered gates against measured values. Prompt spread is *not* reported as zero: the grid held one prompt, so that axis was never exercised.

Artifact. A harness that only ever returns PASS cannot be distinguished from one that is not looking, so the release ships the blocked scorecard, raw cells and digests: loop, cases, generators, gate and `scripts/independent_audit.py` as CHIA-compatible blocks (BSD-3-Clause), <https://github.com/ListenJ/chia-repo-audit>. Reproduce with `python3 chia_loop/loops/audit_repro.py --version 6 --config .tmp/cfg/grid.json --backend champsim_node`.

4 Limitations

Prompt sensitivity is unmeasured, not zero: the grid held one prompt constant; a 3×3 prompt $\times$ seed factorial is generated and queued. **Repeats:** 0.0000% is $N = 2$ per cell and supports no distributional claim; a 5-repeat replication is running. **Annotators:** both models come from one provider, so we claim model-model, not cross-family, independence, and 4 semantic cases are few. **Checker:** the cell-level nullity rule conflates per-trace with global nullity. **Provenance:** `raw.json` records HEAD at write time, not stage entry. **Scope:** one simulator, one design family, three traces; no RTL flow and no gem5 result is claimed.

5 Conclusion

We released an audit loop that separates four variance sources, gates publication on measurement *and* annotation, and keeps its evidence. Against the official CHIA ChampSim image it blocked its own reproducibility claim at 78.16% cross-seed CV, showed seeds moving artifacts without moving designs, showed trace choice reordering candidates ($\tau = 0.556$), and showed two independent language-model auditors agreeing and both wrong on a measured simulator escape. The instrument and the negative verdicts are the contribution.

AI Assistance Disclosure

AI coding and writing tools assisted with implementation, experiment plumbing, and drafting and editing this paper. The human authors are responsible for all code, measurements, citations, claims, and the final content.

References

- [1] R. Gupta et al. ArchAgent: Agentic AI-driven Computer Architecture Discovery. arXiv:2602.22425, 2026.
- [2] CHIA framework. <https://github.com/ucb-bar/chia>. arXiv:2606.27350.
- [3] Meerkat: Cheating Agents. <https://debugml.github.io/cheating-agents/>, 2026.
- [4] NeurIPS 2026 Call for Reproducibility. <https://neurips.cc/Conferences/2026/CallForReproducibility>.
- [5] Auto Benchmark Audit. arXiv:2605.26079, 2026.
- [6] Nourbakhsh et al. GEM 2026 Outstanding Paper. <https://aclanthology.org/2026.gem-main.50/>.